Cheatsheet: comandos y skills de Claude Code

Referencia rápida de comandos slash, skills incluidos y cómo crear los tuyos. Basada en la documentación oficial de Claude Code (code.claude.com/docs); los comandos cambian con las versiones, así que usa `/help` o escribe `/` para ver los de tu instalación.

Índice

- 1. Cómo funcionan
- 2. Sesión y contexto
- 3. Modelo, razonamiento y modos
- 4. Proyecto, memoria y configuración
- 5. Extensiones: MCP, plugins, hooks y agentes
- 6. Revisión de código y Git
- 7. Skills incluidos (bundled)
- 8. Crear tus propios skills
- 9. Cuenta, uso y utilidades
- 10. Fuentes

1. Cómo funcionan

- Un comando solo se reconoce **al principio del mensaje**; el texto que sigue son sus argumentos.
- Escribe `/` para ver el menú y filtra con letras. Si envías un comando mientras Claude responde, se encola hasta que acabe el turno (salvo `/status`, `/tasks`, `/usage`).
- Se pueden encadenar skills en un mensaje: `/skill-a /skill-b haz X` (hasta seis).
- **Comando integrado** (`/help`, `/compact`): lógica fija. **Skill** (`/code-review`, `/batch`): un prompt que Claude orquesta con sus herramientas.
- Un skill tuyo con el mismo nombre sustituye al comando incluido.

2. Sesión y contexto

Comando	Qué hace
`/clear [nombre]`	Nueva conversación con contexto vacío (el nombre etiqueta la anterior).
`/compact [instrucciones]`	Resume la conversación para liberar contexto; puedes indicar en qué centrarse.
`/context [all]`	Muestra el uso de contexto en una cuadrícula y sugerencias.
`/autocompact [auto/tokens]`	Fija cuánto se llena el contexto antes de compactar solo.
`/resume [sesión]`	Retoma una conversación por ID o nombre, o abre el selector.
`/rename [nombre]`	Renombra la sesión (sin nombre, lo genera).
`/branch [nombre]`	Bifurca la conversación para probar otra dirección sin perder la original.
`/fork [prompt]`	Copia la conversación en una sesión en segundo plano y sigues aquí.
`/subtask <tarea>`	Lanza un subagente en segundo plano que hereda la conversación.
`/background [prompt]`	Desacopla la sesión como agente en segundo plano.
`/rewind`	Vuelve a un punto anterior (conversación y/o código). Alias: `/checkpoint`, `/undo`.
`/recap`	Resumen de una línea de la sesión.
`/btw [pregunta]`	Pregunta lateral sin añadirla a la conversación.
`/copy [N]`	Copia la última respuesta (o la N-ésima más reciente).
`/export [fichero]`	Exporta la conversación como texto.

Comando	Qué hace
`/cd <ruta>` / `/add-dir <ruta>`	Cambia de directorio de trabajo / añade uno extra.
`/tasks`	Gestiona el trabajo en segundo plano. Alias: `/bashes`.
`/exit`	Sale del CLI. Alias: `/quit`.

3. Modelo, razonamiento y modos

Comando	Qué hace
`/model [modelo]`	Cambia de modelo y lo guarda como predeterminado.
`/effort [low…max/auto/status]`	Nivel de razonamiento (`low`, `medium`, `high`, `xhigh`, `max`).
`/fast [on/off]`	Activa o desactiva el modo rápido.
`/plan [descripción]`	Entra en modo plan (analiza antes de editar).
`/goal [condición/clear]`	Claude sigue trabajando entre turnos hasta cumplir la condición.
`/advisor [modelo/off]`	Activa un segundo modelo que asesora en momentos clave.
`/sandbox`	Activa o desactiva el modo sandbox (donde esté disponible).
`/permissions`	Reglas allow/ask/deny de herramientas.
`/output-style [estilo]`	Cambia el estilo de salida.

4. Proyecto, memoria y configuración

Comando	Qué hace
`/init`	Genera un `CLAUDE.md` inicial del proyecto.
`/memory`	Edita ficheros `CLAUDE.md` y gestiona la memoria automática.
`/config [clave=valor]`	Abre ajustes (tema, modelo, estilo…) o fija un valor directo.
`/status`	Versión, modelo, cuenta y conectividad.
`/statusline`	Configura la línea de estado.
`/theme`, `/color`, `/tui`	Aspecto: tema, color de la barra, renderizador de terminal.
`/keybindings`	Abre el fichero de atajos.
`/terminal-setup`	Instala Shift+Enter para saltos de línea en tu terminal.
`/ide`	Gestiona integraciones con IDE.
`/diff`	Revisa los cambios del árbol de trabajo.

5. Extensiones: MCP, plugins, hooks y agentes

Comando	Qué hace
`/mcp [reconnect/enable/disable]`	Gestiona servidores MCP y su autenticación OAuth.
`/plugin [list/install/enable/disable]`	Gestiona plugins.
`/reload-plugins`	Recarga plugins sin reiniciar.
`/hooks`	Muestra la configuración de hooks.
`/agents`	Recuerda cómo crear/gestionar subagentes (`.claude/agents/`).
`/list-agents`	Lista subagentes y sesiones a las que Claude puede escribir.
`/skills`	Lista los skills disponibles (filtra escribiendo).
`/reload-skills`	Vuelve a escanear skills y comandos añadidos durante la sesión.
`/skill-doctor`	Coste en contexto y uso de cada skill (para desactivar los inútiles).
`/workflows`	Vista de progreso de workflows en ejecución.
`/schedule [descripción]`	Crea, lista o ejecuta rutinas programadas en la nube.

Comando	Qué hace
`/loop [intervalo] [prompt]`	Repite un prompt mientras la sesión siga abierta.

6. Revisión de código y Git

Comando	Qué hace
`/code-review [nivel] [--fix] [--comment] [pr#/rama/ruta]`	Revisa el diff, un PR, rama o ruta. Alias: `/review`.
`/security-review`	Busca vulnerabilidades en los cambios de la rama.
`/ultrareview [PR/rama]`	Revisión profunda multiagente en sandbox en la nube.
`/simplify [objetivo]`	Revisa el código cambiado en busca de limpieza y aplica arreglos.
`/autofix-pr [prompt]`	Sesión en la nube que vigila tu PR y corrige fallos de CI o comentarios.
`/install-github-app`	Instala la app de GitHub y, opcionalmente, workflows de Actions.

7. Skills incluidos (bundled)

Skill	Para qué sirve
`/batch <instrucción>`	Cambios grandes en paralelo: divide el trabajo en 5–30 unidades independientes y presenta un plan.
`/debug [descripción]`	Activa el log de depuración y diagnostica leyéndolo.
`/doctor [prompt-audit]`	Chequeo de instalación, ajustes, extensiones y `CLAUDE.md`.
`/verify` y `/run`	Arrancan tu app y comprueban que el cambio funciona de verdad, más allá de los tests.
`/run-skill-generator`	Enseña a `/run` y `/verify` a lanzar tu proyecto (crea un skill por proyecto).
`/claude-api [migrate/upgrade/prompt-audit/cost-optimize/build-eval/hillclimb…]`	Referencia y tareas de la API de Claude.
`/claude-in-chrome [tarea]`	Ejecuta una tarea en tu navegador con Claude in Chrome.
`/deep-research <pregunta>`	Workflow de búsqueda, contraste de fuentes e informe citado.
`/fewer-permission-prompts`	Propone una allowlist a partir de tus transcripciones para pedir menos permisos.
`/update-config [petición]`	Edita ajustes (permisos, variables, hooks) según lo que describas.
`/dataviz`, `/design`, `/slides`	Guías para gráficos, diseños/maquetas y presentaciones como artifacts.

8. Crear tus propios skills

Ubicaciones (carpeta con un `SKILL.md`):

Ámbito	Ruta
Personal (todos tus proyectos)	`~/.claude/skills/<nombre>/SKILL.md`
Proyecto (compartible por Git)	`.claude/skills/<nombre>/SKILL.md`
Plugin	`<plugin>/skills/<nombre>/SKILL.md` → `/plugin:skill`

Los ficheros antiguos `.claude/commands/deploy.md` siguen funcionando y crean `/deploy`. Los cambios en skills existentes se recogen sin reiniciar; para un directorio nuevo usa `/reload-skills`.

Ejemplo mínimo:

```
---
name: deploy
description: Despliega la aplicación a producción
```

```
disable-model-invocation: true
allowed-tools: Bash(git add *) Bash(git commit *)
---

Despliega $ARGUMENTS a producción:
1. Ejecuta los tests
2. Compila la aplicación
```

Campos del frontmatter (todos opcionales; se recomienda `description`):

Campo	Función
`name`	Nombre del comando; por defecto, el de la carpeta.
`description` / `when_to_use`	Qué hace y cuándo usarlo; Claude decide con esto si lo carga.
`argument-hint`	Pista de autocompletado, p. ej. `[issue]`.
`arguments`	Argumentos con nombre para `$nombre`.
`disable-model-invocation`	`true`: solo lo invocas tú.
`user-invocable`	`false`: oculto del menú, solo lo usa Claude.
`allowed-tools`	Herramientas preaprobadas durante el turno (no restringe las demás).
`disallowed-tools`	Herramientas retiradas mientras el skill está activo.
`model` / `effort`	Modelo y nivel de razonamiento del turno.
`context: fork` + `agent`	Ejecuta el skill en un subagente aislado (`Explore`, `Plan`, `general-purpose` o uno propio).
`paths`	Globs: se carga solo al trabajar con ficheros que coincidan.
`hooks`	Hooks registrados al invocar el skill.
`shell`	`bash` (defecto) o `powershell` para la inyección ``!`cmd```.

Quién lo invoca:

Ajuste	Tú	Claude
(por defecto)	Sí	Sí
`disable-model-invocation: true`	Sí	No
`user-invocable: false`	No	Sí

Sustituciones: `$ARGUMENTS` (todos), `$0`/`$1` o `$ARGUMENTS[N]` (por posición, empieza en 0; entrecomilla los de varias palabras), `$nombre`, `${CLAUDE_SESSION_ID}`, `${CLAUDE_SKILL_DIR}`, `${CLAUDE_PROJECT_DIR}`, `${CLAUDE_EFFORT}`. Escapa un dólar literal con `\$`.

Contexto dinámico: una línea ``!`git diff HEAD`` ejecuta el comando antes de enviar el skill y Claude recibe su salida. Si el comando falla (código distinto de 0), se aborta el skill; añade `|| true` si es aceptable. Preaprueba con `allowed-tools: Bash(gh *)`.

Buenas prácticas: mantén `SKILL.md` por debajo de unas 500 líneas, mueve el detalle a ficheros de apoyo enlazados, pon el caso de uso principal al principio de `description` y usa `context: fork` para tareas con mucha salida.

9. Cuenta, uso y utilidades

Comando	Qué hace
`/help`	Ayuda y comandos disponibles.
`/usage`	Coste de la sesión, límites del plan y estadísticas. Alias: `/cost`, `/stats`.
`/login`, `/logout`	Inicia o cierra sesión.
`/release-notes`	Changelog por versiones.
`/insights`	Informe HTML sobre cómo usas Claude Code.

Comando	Qué hace
`/bug`, `/feedback`	Informa de un fallo o envía comentarios.
`/desktop`, `/teleport`, `/remote-control`	Continuar en la app de escritorio, traer una sesión de la nube o controlar la sesión desde claude.ai.
`/powerup`	Lecciones interactivas de funciones.
`/vim`	Eliminado: usa `/config` → modo de edición.

10. Fuentes

- Comandos: https://code.claude.com/docs/en/commands
- Skills: https://code.claude.com/docs/en/skills
- La lista refleja la documentación consultada el 5 de noviembre de 2026; algunos comandos dependen de versión, plan o plataforma.